

Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs\

ChatGPT and Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)

Google Chrome

c. Prompts used. (You must use the same prompt for both GenAIs.)

Can you make a C++ program that uses objects and not functions and prioritizes emails for a company CEO:

It will use a MaxHeap as a means of implementing a priority queue. A queue where emails can shift towards the front of the queue based on priority status
The MaxHeap uses a list-based implementation. then having the MaxHeap handle all of the emails prioritizing for the CEO
Functions must be created from scratch DO NOT INCLUDE pre-existing heap modules

I like the program already but lets assume there is a text file being inputted to the program instead of us writing our own emails.

Should have different commands it can read from the text file:

EMAIL: The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT: Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ: CEO has read email and dealt with it

COUNT: Display current count of unread emails

d. Generated code from both GenAIs.

```
//ChatGPT:
https://chatgpt.com/c/6ab1d7e4-af7c-83e9-91df-675ac3587653
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>

using namespace std;

// -----
// EMAIL CLASS
// -----

class Email
{
private:
    string senderCategory;
    string subject;
    string date;

    int priority;

public:

    // Constructor
    Email(string category, string emailSubject, string
emailDate)
    {
        senderCategory = category;
        subject = emailSubject;
        date = emailDate;

        // Assign priority based on sender category
```

```
        if (senderCategory == "Boss")
        {
            priority = 5;
        }
        else if (senderCategory == "Subordinate")
        {
            priority = 4;
        }
        else if (senderCategory == "Peer")
        {
            priority = 3;
        }
        else if (senderCategory == "ImportantPerson")
        {
            priority = 2;
        }
        else
        {
            priority = 1;
        }
    }

    // Get sender category
    string getSenderCategory()
    {
        return senderCategory;
    }

    // Get subject
    string getSubject()
    {
        return subject;
    }

    // Get date
    string getDate()
    {
        return date;
    }
}
```

```

        // Get priority
        int getPriority()
        {
            return priority;
        }

        // Display email information
        void display()
        {
            cout << "Sender: " << senderCategory << endl;
            cout << "Subject: " << subject << endl;
            cout << "Date: " << date << endl;
        }
    };

    // -----
    // MAXHEAP CLASS
    // -----

class MaxHeap
{
private:

    // List-based implementation of the heap
    vector<Email> heap;

    // Compare two emails
    // Returns true if email1 should come before email2
    bool hasHigherPriority(Email email1, Email email2)
    {
        // Compare sender categories first
        if (email1.getPriority() > email2.getPriority())
        {
            return true;
        }

        if (email1.getPriority() < email2.getPriority())
        {

```

```
        return false;
    }

    // If sender categories are equal,
    // compare the dates.
    // Newest date gets higher priority.

    int year1 = stoi(email1.getDate().substr(6, 4));
    int month1 = stoi(email1.getDate().substr(0, 2));
    int day1 = stoi(email1.getDate().substr(3, 2));

    int year2 = stoi(email2.getDate().substr(6, 4));
    int month2 = stoi(email2.getDate().substr(0, 2));
    int day2 = stoi(email2.getDate().substr(3, 2));

    if (year1 > year2)
    {
        return true;
    }

    if (year1 < year2)
    {
        return false;
    }

    if (month1 > month2)
    {
        return true;
    }

    if (month1 < month2)
    {
        return false;
    }

    if (day1 > day2)
    {
        return true;
    }
}
```

```

        return false;
    }

    // Move an email upward in the heap
    void heapifyUp(int index)
    {
        while (index > 0)
        {
            int parent = (index - 1) / 2;

            if (hasHigherPriority(heap[index], heap[parent]))
            {
                Email temporary = heap[index];

                heap[index] = heap[parent];

                heap[parent] = temporary;

                index = parent;
            }
            else
            {
                break;
            }
        }
    }

    // Move an email downward in the heap
    void heapifyDown(int index)
    {
        int size = heap.size();

        while (true)
        {
            int leftChild = (2 * index) + 1;
            int rightChild = (2 * index) + 2;

            int largest = index;

```

```

        // Check left child
        if (leftChild < size &&
            hasHigherPriority(heap[leftChild],
heap[largest]))
        {
            largest = leftChild;
        }

        // Check right child
        if (rightChild < size &&
            hasHigherPriority(heap[rightChild],
heap[largest]))
        {
            largest = rightChild;
        }

        // If the current item is already largest,
        // the heap is in the correct order.
        if (largest == index)
        {
            break;
        }

        Email temporary = heap[index];

        heap[index] = heap[largest];

        heap[largest] = temporary;

        index = largest;
    }
}

public:

    // Add an email to the MaxHeap
    void insert(Email email)
    {

```

```

        heap.push_back(email);

        int lastIndex = heap.size() - 1;

        heapifyUp(lastIndex);
    }

    // Check if the heap is empty
    bool isEmpty()
    {
        return heap.empty();
    }

    // Return the highest-priority email
    // without removing it
    Email peek()
    {
        return heap[0];
    }

    // Remove the highest-priority email
    void removeMax()
    {
        if (heap.empty())
        {
            return;
        }

        // Move the last email to the root
        heap[0] = heap[heap.size() - 1];

        // Remove the last item
        heap.pop_back();

        // Restore the MaxHeap
        if (!heap.empty())
        {

```

```

        heapifyDown(0);
    }
}

// Return the number of emails
int getCount()
{
    return heap.size();
}
};

// -----
// CEO INBOX CLASS
// -----

class CEOInbox
{
private:
    MaxHeap emailQueue;

    // Process an EMAIL command
    void processEmail(string emailInformation)
    {
        string category;
        string subject;
        string date;

        stringstream input(emailInformation);

        // Read sender category
        getline(input, category, ',');

        // Read subject
        getline(input, subject, ',');

        // Read date
        getline(input, date);
    }
};

```

```
// Create an Email object
Email newEmail(category, subject, date);

// Insert email into the MaxHeap
emailQueue.insert(newEmail);
}

// Process a NEXT command
void processNext()
{
    if (emailQueue.isEmpty())
    {
        cout << "No unread emails." << endl;
        return;
    }

    // Display the highest-priority email
    // without removing it
    emailQueue.peek().display();
}

// Process a READ command
void processRead()
{
    if (emailQueue.isEmpty())
    {
        cout << "No unread emails to read." << endl;
        return;
    }

    // Remove the highest-priority email
    emailQueue.removeMax();

    cout << "Email has been read." << endl;
}
```

```

// Process a COUNT command
void processCount()
{
    cout << "Unread emails: "
          << emailQueue.getCount()
          << endl;
}

public:

// Read commands from a file
void readFile(string fileName)
{
    ifstream inputFile(fileName);

    if (!inputFile.is_open())
    {
        cout << "Error: Could not open file." << endl;
        return;
    }

    string line;

    // Read the file one line at a time
    while (getline(inputFile, line))
    {
        if (line.empty())
        {
            continue;
        }

        // Find the command
        if (line.substr(0, 5) == "EMAIL")
        {
            // Remove "EMAIL "
            string emailInformation = line.substr(6);

            processEmail(emailInformation);
        }
    }
}

```

```

        else if (line == "NEXT")
        {
            processNext();
        }
        else if (line == "READ")
        {
            processRead();
        }
        else if (line == "COUNT")
        {
            processCount();
        }
        else
        {
            cout << "Unknown command: "
                 << line
                 << endl;
        }
    }

    inputFile.close();
}

};

// -----
// MAIN PROGRAM
// -----

int main()
{
    CEOInbox inbox;

    inbox.readFile("emails.txt");

    return 0;
}

```

```
//Gemni: https://gemini.google.com/app/1e4a1a92268adad3
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

// Custom Dynamic List to manage heap elements internally
template <typename T>
class CustomList {
private:
    T* data;
    int capacity;
    int currentSize;

    void resize() {
        capacity *= 2;
        T* newData = new T[capacity];
        for (int i = 0; i < currentSize; ++i) {
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
    }

public:
    CustomList() : capacity(10), currentSize(0) {
        data = new T[capacity];
    }

    ~CustomList() {
        delete[] data;
    }

    void append(const T& element) {
        if (currentSize == capacity) {
            resize();
        }
        data[currentSize++] = element;
    }
}
```

```

void removeLast() {
    if (currentSize > 0) {
        currentSize--;
    }
}

T& operator[](int index) {
    return data[index];
}

const T& operator[](int index) const {
    return data[index];
}

int size() const {
    return currentSize;
}

bool empty() const {
    return currentSize == 0;
}
};

// Class representing an individual Email
class Email {
private:
    std::string senderCategory;
    std::string subject;
    std::string date; // Format: MM-DD-YYYY

    // Helper to extract numerical priority from sender
    category
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        return 1; // "OtherPerson" or any default
    }
}

```

```

        // Converts MM-DD-YYYY to YYYYMMDD string for easy
        chronological comparison
        std::string getNormalizedDate() const {
            if (date.length() != 10) return date;
            std::string month = date.substr(0, 2);
            std::string day = date.substr(3, 2);
            std::string year = date.substr(6, 4);
            return year + month + day;
        }

public:
    Email() = default;
    Email(std::string sender, std::string subj, std::string
dt)
        : senderCategory(sender), subject(subj), date(dt) {}

    std::string getSender() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return date; }

    // Priority Comparison Operators
    // Returns true if this email has HIGHER priority than
other
    bool operator>(const Email& other) const {
        int p1 = getCategoryPriority();
        int p2 = other.getCategoryPriority();

        if (p1 != p2) {
            return p1 > p2;
        }

        // Tie-breaker: Newer date takes precedence (Sprint
management rule)
        return getNormalizedDate() >
other.getNormalizedDate();
    }

    bool operator<(const Email& other) const {
        return other > *this;
    }

```

```

};

// Custom List-based MaxHeap implementation
class MaxHeap {
private:
    CustomList<Email> heap;

    void swap(int i, int j) {
        Email temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heap[index] > heap[parentIndex]) {
                swap(index, parentIndex);
                index = parentIndex;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int maxIndex = index;
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;

        if (leftChild < heap.size() && heap[leftChild] >
heap[maxIndex]) {
            maxIndex = leftChild;
        }

        if (rightChild < heap.size() && heap[rightChild] >
heap[maxIndex]) {
            maxIndex = rightChild;
        }
    }
}

```

```

        if (index != maxIndex) {
            swap(index, maxIndex);
            heapifyDown(maxIndex);
        }
    }

public:
    void push(const Email& email) {
        heap.append(email);
        heapifyUp(heap.size() - 1);
    }

    Email peek() const {
        if (heap.empty()) {
            throw std::runtime_error("Heap is empty!");
        }
        return heap[0];
    }

    void pop() {
        if (heap.empty()) return;

        heap[0] = heap[heap.size() - 1];
        heap.removeLast();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    int getSize() const {
        return heap.size();
    }

    bool isEmpty() const {
        return heap.empty();
    }
};

// CEO Email Inbox Processor Object
class CEOInboxProcessor {

```

```

private:
    MaxHeap priorityQueue;

    void processEmailCommand(std::istream& ss) {
        std::string sender, subject, date;

        if (ss >> sender >> subject >> date) {
            Email newEmail(sender, subject, date);
            priorityQueue.push(newEmail);
        }
    }

    void processNextCommand() const {
        if (priorityQueue.isEmpty()) {
            std::cout << "No unread emails in the inbox.\n\n";
            return;
        }

        Email topEmail = priorityQueue.peek();
        std::cout << "Sender: " << topEmail.getSender() <<
"\n";
        std::cout << "Subject: " << topEmail.getSubject() <<
"\n";
        std::cout << "Date: " << topEmail.getDate() << "\n\n";
    }

    void processReadCommand() {
        if (priorityQueue.isEmpty()) {
            std::cout << "No email to read.\n\n";
            return;
        }

        priorityQueue.pop();
    }

    void processCountCommand() const {
        std::cout << "Unread emails: " <<
priorityQueue.getSize() << "\n\n";
    }

```

```

public:
    void executeCommandsFromFile(const std::string& filename)
    {
        std::ifstream file(filename);

        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " <<
filename << std::endl;
            return;
        }

        std::string line;
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            std::istringstream ss(line);
            std::string command;
            ss >> command;

            if (command == "EMAIL") {
                processEmailCommand(ss);
            } else if (command == "NEXT") {
                processNextCommand();
            } else if (command == "READ") {
                processReadCommand();
            } else if (command == "COUNT") {
                processCountCommand();
            }
        }

        file.close();
    }
};

int main(int argc, char* argv[]) {
    std::string filename = "commands.txt";
    if (argc > 1) {
        filename = argv[1];
    }
}

```

```

CEOInboxProcessor inbox;
inbox.executeCommandsFromFile(filename);

return 0;
}

```

- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
 - i. **Correctness:** In terms of correctness of the code both output the correct outputs. But there are some risks with these programs. ChatGPT works perfectly with the inputted file. While Gemini has trouble whenever the file has spaces which could cause errors.
 - ii. **Execution Time:** For execution time they both are equal since each operation used has the same $O()$ efficiency.
 - iii. **Space Complexity:** For the space complexity they are essentially the same. But ChatGPT manages its own memory so it doesn't need to resize itself. Gemini manually sizes the memory so it's set to a certain number.
 - iv. **Maintainability:** Both Ai's have decent comments but I will agree that ChatGPTs code is easier to understand in terms of structure. Gemini is harder to read even though it has fewer lines of code.
- f. Select one to use as the basis for this assignment and justify why you picked it.
 - i. I am going to use ChatGPT's code since the execution time is essentially the same so there is no slower code technically. Also it is just simpler code I can understand and upgrade to make it better.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
 - i. **Correctness:** In terms of correctness the input is correct but I will edit the input to make sure it equals the example. I will also make sure the code can read the txt with extra spaces.
 - ii. **Execution Time:** I wouldn't change the execution time since the efficiency is already great.
 - iii. **Space Complexity:** Since the chatGPT code manages its own memory I believe this is efficient enough.
 - iv. **Maintainability:** Will add more comments to make sure each line is understood and I added a feature that allows the user to input their own files.